PRISMSPACE - INTELLIGENT MULTI-AGENT PLATFORM



Dept. of Computer Science & Engineering
College of Engineering, Karunagappally

Project Group 01

NOBIN SIJO	KNP23CS082
PRANAV P	KNP23CS085
PRASANTH P	KNP23CS086
SANKAR NARAYAN S	KNP23CS094

Project Coordinator & Guide
Dr. Leena Silvoster
Associate Professor
College of Engineering, Karunagappally

01. Introduction: PrismSpace Platform

- **Versatile Multi-Agent Platform:** A local-first workspace that coordinates specialized AI (Artificial Intelligence) agents not solely for coding or research, but also for **day-to-day tasks** (email triage, document processing, daily scheduling, file organization, note synthesis) with shared context awareness.
- **Dynamic DAG Task Planning:** Converts natural-language user queries into structured Directed Acyclic Graphs (DAGs) for dependency-aware, parallel sub-task execution.
- **Standardized Tool Integration via MCP:** Equips agents with practical everyday and developer tools (Gmail, Google Drive, Filesystem, Terminal CLI, GitHub, Browser) using the open Model Context Protocol (MCP).
- **Context-Aware Multi-Provider Routing:** Intelligently directs sub-tasks across Groq, OpenAI, and NVIDIA NIM (NVIDIA Inference Microservices) with human-in-the-loop approval checkpoints and ORPO (Odds Ratio Preference Optimization) alignment.
- **Reactive Local Persistence:** Powered by Dexie.js (`PrismDB`) on browser IndexedDB (Indexed Database), providing real-time Server-Sent Events (SSE) telemetry and zero-latency client state synchronization.

02. Project Objectives

1. **Context-Aware Task Planning:** Convert everyday user goals (communication, file management, daily schedules) and technical requests into structured Directed Acyclic Graphs (DAGs) for organized, dependency-safe execution.

2. **Intelligent Model & Agent Routing:** Dynamically route each sub-task to the most suitable Large Language Model (LLM) backend and dispatch specialized, context-aware agents in parallel.

3. **Practical Tool Execution via MCP:** Connect agents to everyday productivity tools and development utilities (Gmail, Google Drive, Filesystem, Terminal CLI, GitHub) through the Model Context Protocol (MCP), accompanied by live Server-Sent Events (SSE) updates.

4. **Safety Verification & Local Persistence:** Validate outputs with human approval checkpoints and Odds Ratio Preference Optimization (ORPO), storing results reactively in local IndexedDB (Indexed Database: `PrismDB`).

03. PrismSpace Workflow (Figure 1.0)

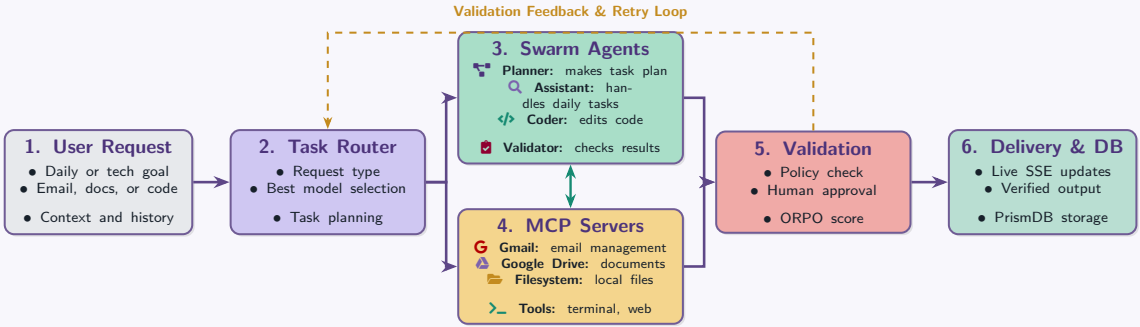


Figure 1.0: PrismSpace Workflow | **DAG** = Directed Acyclic Graph | **MCP** = Model Context Protocol | **ORPO** = Odds Ratio Preference Optimization | **SSE** = Server-Sent Events

04. System Design Principles

Architectural Design Foundations

- **Layered Decoupling:** Strict separation between React 19 Client UI (User Interface), FastAPI Python Hive Bridge, and Foundation LLMs (Large Language Models).
- **Dynamic DAG Scheduling:** Task dependencies represented as Directed Acyclic Graphs (DAGs) $\mathcal{G}(V, E)$. Sub-tasks run in parallel ($K \leq 5$) for both daily productivity and technical workflows.
- **Standardized Protocol (MCP):** Replaces custom plugins with open Model Context Protocol (MCP) JSON-RPC (JavaScript Object Notation Remote Procedure Call) contracts.
- **Human-in-the-Loop Checkpoints:** Critical operations (sending emails, modifying files, executing terminal commands) require explicit user approval.

Data & Runtime Design

- **Local IndexedDB (`PrismDB`):** ACID-compliant (Atomicity, Consistency, Isolation, Durability) browser storage for chat sessions, daily tasks, tool history, and files.
- **Reactive Subscriptions:** Dexie `useLiveQuery()` auto-updates UI components on database change events without network polling.
- **Server-Sent Events (SSE):** Asynchronous token delivery and agent telemetry stream over persistent HTTP (Hypertext Transfer Protocol).
- **Calibrated Preference Gating:** Odds Ratio Preference Optimization (ORPO) ensures outputs adhere to human quality and safety standards.

Design Philosophy: Versatile multi-agent workspace for everyday tasks and engineering: context awareness, sandboxed safety, and real-time observability.

05. Proposed Methods

1. Multi-Head Routing

Sparse-Dense Feature Fusion:

- 10k TF-IDF (Term Frequency–Inverse Document Frequency) n-grams + lexical metrics.
- 12-Class Intent Classifier identifying user task type (daily tasks, emails, docs, coding, research).
- Softmax Provider Head selecting optimal LLM (Large Language Model) backend.
- Sigmoid Multi-Label Dispatcher activating required swarm agents.

2. Swarm DAG & ReAct Loop

Autonomous Collaboration:

- **DAG (Directed Acyclic Graph):** Topological sort computes dependency execution order.
- **ReAct (Reasoning & Acting):** Agents alternate Observation $\rightarrow$ Thought $\rightarrow$ Action Matrix $\mathbf{A}_{i,j}$.
- **MCP (Model Context Protocol):** Invokes tools (Gmail, Drive, Filesystem, Terminal CLI, GitHub).

3. ORPO & Local DB

Safety & Persistence:

- **Single-Stage ORPO:** Odds Ratio Preference Optimization: $\mathcal{L}_{\text{ORPO}} = \mathcal{L}_{\text{SFT}} + \beta \mathcal{L}_{\text{OR}}$ with SFT (Supervised Fine-Tuning).
- **Safety Checkpoint Gate:** Threshold-tuned log-odds filter against prompt injection & unsafe actions.
- **Dexie `PrismDB`:** Reactive IndexedDB (Indexed Database) storage with `useLiveQuery()`.

Pipeline Synergy: Inputs flow from feature extraction $\rightarrow$ DAG scheduling $\rightarrow$ ReAct tool execution $\rightarrow$ ORPO safety validation $\rightarrow$ reactive client delivery.

07. Complete Explanation: Flow Stages 1, 2, and 3

Stage 1: Ingestion & Features

- **Input Ingestion:** Captures requests for day-to-day productivity (email drafting, file organization, note synthesis) and technical engineering.
- **Tokenization & Cleaning:** Removes malformed tokens, strips control characters, normalizes whitespace.
- **Feature Extraction:** Constructs 10k TF-IDF (Term Frequency–Inverse Document Frequency) vectors + 5D lexical metrics.
- **Imputation & Outliers:** Fills missing metadata and filters out-of-distribution prompts via IsolationForest.

Stage 2: Intelligence & DAG

- **12-Class Intent Classifier:** Identifies user task type across daily assistance, emails, documents, coding, and research.
- **Provider Router:** Directs sub-tasks to best LLM backend (Groq for rapid loops, OpenAI for reasoning, NVIDIA NIM).
- **Swarm Dispatcher:** Activates context-aware specialized agents ($K \leq 5$).
- **DAG Construction:** Builds Directed Acyclic Graph (DAG) $\mathcal{G}(V, E)$ for parallel, dependency-aware execution.
- **Safety Gate:** Assesses risk before launching sensitive operations.

Stage 3: Execution & Storage

- **ReAct MCP Loop:** Workers execute Observation $\rightarrow$ Thought $\rightarrow$ Action using Model Context Protocol (MCP) tools.
- **SSE Streaming:** Emits tokens and logs live via Server-Sent Events (SSE).
- **ORPO Evaluation:** Odds Ratio Preference Optimization verifies deliverables against user preferences and safety.
- **IndexedDB (`PrismDB`):** Commits run history atomically via Dexie.js for instant UI re-rendering.

Data Flow Transition: Clean feature vectors in Stage 1 drive deterministic planning in Stage 2, which directs sandboxed execution and reactive persistence in Stage 3.

08. PrismSpace System Architecture & Swarm Workflow (Figure 1.2)

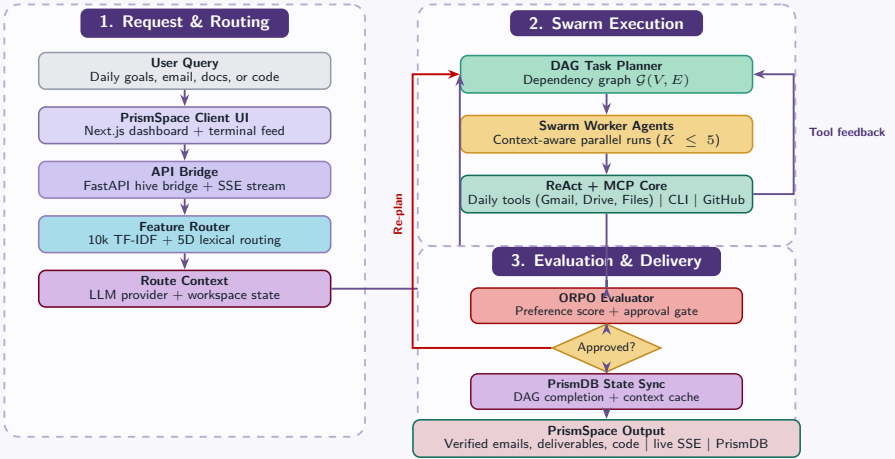


Figure 1.2: Architecture & Swarm Workflow | **DAG** = Directed Acyclic Graph | **ReAct** = Reasoning & Acting | **MCP** = Model Context Protocol | **ORPO** = Preference Optimization | **SSE** = Server-Sent Events

09. Browser Database Schema: PrismDB (Figure 1.3)

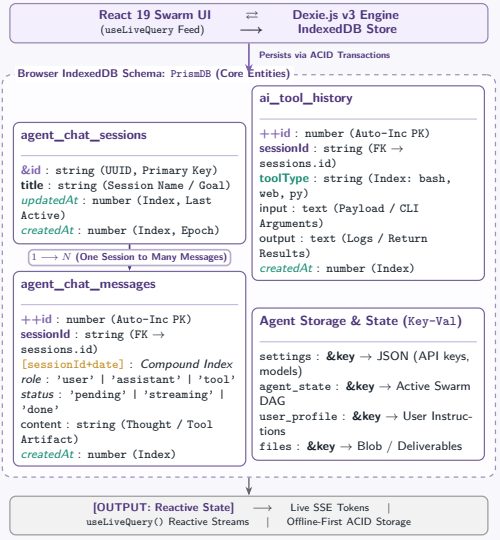


Figure 1.3: Browser IndexedDB Database Schema and Reactive Data Architecture (PrismDB)

Glossary

1. PrismDB
Browser IndexedDB (Indexed Database) storing multi-agent chats, daily tasks & tool telemetry.

2. Dexie.js v3
TypeScript IDB wrapper handling schema migrations & ACID (Atomicity, Consistency, Isolation, Durability) transactions.

3. PK & FK Keys
Primary Key (PK: `&id` UUID, `++id` Auto-Inc) and Foreign Key (FK) relational links to sessions.

4. Compound Idx
B-tree index `[session+date]` for instant sub-millisecond query slicing.

5. `useLiveQuery()`
Reactive hook auto-rerendering React UI (User Interface) components on local DB changes.

Algorithm: Step 1 – Understand the User Prompt

Step 1: Prompt Understanding

1. Read the request and identify the user's main goal.
2. Extract constraints, files, preferences, deadlines, and expected results.
3. Add relevant conversation history and project context.
4. Detect whether the request needs tools, human approval, or extra clarification.
5. Create a structured task summary for the router.

Methods used: natural language understanding, context retrieval, intent and constraint extraction.

Algorithm: Step 2 – Classify the Request

Step 2: Intent and Requirement Analysis

1. Classify the request as an everyday, document, research, coding, or mixed task.
2. Identify required capabilities such as reading, writing, searching, or communication.
3. Estimate task complexity, urgency, and risk.
4. Mark actions that are read only, reversible, or sensitive.

Methods used: intent classification, keyword and context matching, rule based safety screening.

Algorithm: Step 3 – Plan and Route the Task

Step 3: Task Decomposition and Model Routing

1. Break the request into smaller dependent or independent tasks.
2. Build a Directed Acyclic Graph (DAG) showing task order and dependencies.
3. Select the best agent for each task: Planner, Assistant, Researcher, Coder, or Reviewer.
4. Route each task to a suitable model based on reasoning need, speed, and cost.
5. Run independent tasks in parallel when possible.

Key equation: $\text{Model}^* = \arg\min_m(\text{cost}_m + \text{latency}_m)$ subject to task quality. **Methods used:** DAG planning, topological ordering, multi provider model routing, parallel dispatch.

Algorithm: Step 4 – Select the Agent Team

Step 4: Swarm Agent Selection

1. The Planner Agent organizes the task and tracks dependencies.
2. The Assistant Agent handles email, documents, schedules, and daily tasks.
3. The Researcher Agent gathers and summarizes relevant information.
4. The Coder Agent writes, edits, and tests code.
5. The Reviewer Agent checks intermediate work and sends feedback for correction.

Methods used: role based agent dispatch, capability matching, concurrent swarm execution, shared context.

Algorithm: Step 5 – Execute with MCP Tools

Step 5: ReAct Reasoning and MCP Tool Use

1. The agent observes the current task state and decides the next action.
2. It calls the required MCP server: Gmail, Google Drive, Filesystem, Terminal, or Web.
3. The tool result is returned to the agent as new context.
4. The agent continues until the assigned subtask is complete.
5. Sensitive actions pause at the human approval checkpoint.

Core loop: Observation → Thought → Action → Observation. **Methods used:** ReAct loop, MCP JSON RPC tool calls, tool result feedback, human in the loop approval.

Algorithm: Step 6 – Share Context and Memory

Step 6: Context Synchronization

1. Pass completed subtask results to the next dependent agent.
2. Keep the original request, constraints, tool results, and decisions together.
3. Remove duplicate or outdated context before sending the next task.
4. Update the task state, agent status, and completed DAG nodes.
5. Save useful conversation and tool state locally in PrismDB.

Methods used: shared task memory, state synchronization, DAG node updates, Dexie.js and IndexedDB persistence.

Algorithm: Step 7 – Validate and Retry

Step 7: Verification and Safety

1. The Reviewer or Validator Agent checks correctness and completeness.
2. Check that the response follows the user's request and project constraints.
3. Check tool actions for unsafe commands, unauthorized access, or policy violations.
4. Use human approval for high impact actions.
5. If verification fails, return the task to the relevant agent with corrective feedback.

Key equation: Accept when quality score $\geq \tau_{\text{safety}}$. **Methods used:** rule based validation, completeness checking, human approval gates, ORPO preference scoring, retry loop.

Algorithm: Step 8 – Deliver and Monitor

Step 8: Verified Delivery and Monitoring

1. Combine verified subtask results into the final answer or deliverable.
2. Stream progress, tool events, and final results to the interface using SSE.
3. Store the final response, task state, and useful metadata in PrismDB.
4. Record agent latency, tool status, failures, and approval decisions.
5. Use telemetry to improve future routing and task execution.

Methods used: Server Sent Events, reactive UI updates, IndexedDB persistence, telemetry collection, post task review.

10. Conclusion & Key Contributions

Key Technical Contributions

- **Versatile Multi-Agent Orchestration:** End-to-end pipeline handling both everyday productivity (emails, docs, planning) and technical coding.
- **Standardized MCP Architecture:** Model Context Protocol JSON-RPC tool contracts unifying desktop and developer tools.
- **Multi-Provider LLM Optimization:** Balances speed, reasoning depth, and cost across diverse LLM backends.
- **Single-Stage Preference Safety:** Odds Ratio Preference Optimization (ORPO) alignment and calibrated checkpointing.

Impact & System Advantages

- **Full Execution Observability:** Live Server-Sent Events (SSE) stream lets users inspect every reasoning step and tool call.
- **Zero-Delay Client Reactivity:** Browser-native Dexie `PrismDB` updates UI state sub-millisecond without network delays.
- **Controllable Automation:** Human-in-the-loop approval gates prevent unauthorized actions across emails, files, and commands.
- **Context-Aware Extensibility:** Modular agents and MCP servers allow adding new everyday and specialized skills seamlessly.

Conclusion: PrismSpace delivers a versatile, production-ready framework for **controllable, observable, and context-aware Multi-Agent AI across everyday tasks and engineering**.

Thank You!

Questions & Discussion

Nobin Sijo | Pranav P | Prasanth P | Sankar Narayan S

Department of Computer Science & Engineering
College of Engineering, Karunagappally